

【找最小数】

题目描述

给一个正整数NUM1，计算出新正整数NUM2，NUM2为NUM1中移除N位数字后的结果，需要使得NUM2的值最小。

输入描述

- 1.输入的第一行为一个字符串，字符串 由0-9字符组成，记录正整数NUM1，NUM1长度小于32。
- 2.输入的第二行为需要移除的数字的个数，小于NUM1长度。

输出描述

输出一个数字字符串，记录最小值 NUM2。

用例

输入	2615371 4
输出	131
说明	无

题目解析

本题和[算法 - 移除 K 位数字_伏城之外的博客-CSDN博客](#)

相同，但是本题题目描述和用例不清晰，建议看上面[leetcode](#) 算法题后再来写本题

Java算法源码

```
1 import java.util.ArrayDeque;
2 import java.util.Scanner;
3
4 public class Main {
5     public static void main(String[] args) {
6         Scanner sc = new Scanner(System.in);
7
8         String num = sc.next();
9         int n = sc.nextInt();
10
11         System.out.println(solution(num, n));
12     }
13
14     public static String solution(String num, int k) {
15         if (k >= num.length()) {
16             return "0";
17         }
18
19         ArrayDeque<Character> stack = new ArrayDeque<>();
20         int remain = num.length() - k;
21
22         for (int i = 0; i < num.length(); i++) {
23             char c = num.charAt(i);
24
25             while (!stack.isEmpty() && k > 0 && stack.getLast() > c) {
26                 stack.removeLast();
27                 k--;
28             }
29
30             stack.addLast(c);
31         }
32
33         while (stack.size() > remain) {
34             stack.removeLast();
35         }
```

```

36 |
37 |         while (stack.getFirst() == '0' && stack.size() > 1) {
38 |             stack.removeFirst();
39 |         }
40 |
41 |         StringBuilder sb = new StringBuilder();
42 |         for (Character c : stack) {
43 |             sb.append(c);
44 |         }
45 |
46 |         return sb.toString();
47 |     }
48 | }

```

JS算法源码

```

1 | const rl = require("readline").createInterface({ input: process.stdin });
2 | var iter = rl[Symbol.asyncIterator]();
3 | const readline = async () => (await iter.next()).value;
4 |
5 | void (async function () {
6 |     const num = await readline();
7 |     const n = parseInt(await readline());
8 |
9 |     console.log(solution(num, n));
10 | })();
11 |
12 | function solution(num, k) {
13 |     if (k >= num.length) {
14 |         return "0";
15 |     }
16 |
17 |     const stack = [];
18 |     const remain = num.length - k;
19 |

```

运行

```

20   for (let c of num) {
21       while (stack.length > 0 && k > 0 && stack.at(-1) > c) {
22           stack.pop();
23           k--;
24       }
25
26       stack.push(c);
27   }
28
29   while (stack.length > remain) {
30       stack.pop();
31   }
32
33   while (stack[0] == "0" && stack.length > 1) {
34       stack.shift();
35   }
36
37   return stack.join("");
38 }

```

Python算法源码

```

1  # 输入获取
2  num = input()
3  k = int(input())
4
5
6  # 算法入口
7  def solution():
8      global k
9
10     if len(num) <= k:
11         return "0"
12

```

```

13     remain = len(num) - k
14
15     stack = []
16     for c in num:
17         while len(stack) > 0 and k > 0 and stack[-1] > c:
18             stack.pop()
19             k -= 1
20
21         stack.append(c)
22
23     while len(stack) > remain:
24         stack.pop()
25
26     while stack[0] == '0' and len(stack) != 1:
27         stack.pop(0)
28
29     return "".join(stack)
30
31
32 # 算法调用
33 print(solution())

```

C算法源码

```

1  #include <stdio.h>
2  #include <string.h>
3
4  #define MAX_LEN 32
5
6  int main() {
7      char num[MAX_LEN];
8      int k;
9
10     scanf("%s %d", num, &k);
11

```

```
12     int remain = strlen(num) - k;13 |
14     if (remain <= 0) {
15         puts("0");
16         return 0;
17     }
18
19     char stack[MAX_LEN] = {'\0'};
20     int stack_size = 0;
21
22     for (int i = 0; i < strlen(num); i++) {
23         char c = num[i];
24
25         while (stack_size > 0 && k > 0 && stack[stack_size - 1] > c) {
26             stack[--stack_size] = '\0';
27             k--;
28         }
29
30         stack[stack_size++] = c;
31     }
32
33     while (stack_size > remain) {
34         stack[--stack_size] = '\0';
35     }
36
37     int start_index = 0;
38     while (stack[start_index] == '0' && start_index < stack_size - 1) {
39         start_index++;
40     }
41
42     puts(stack + start_index);
43
44     return 0;
45 }
```

C++ 算法源码

```
1  #include <bits/stdc++.h>
2
3  using namespace std;
4
5  string solution(string &num, int k) {
6      if (k >= num.length()) {
7          return "0";
8      }
9
10     int remain = num.length() - k;
11
12     deque<char> stack;
13     for (char c: num) {
14         while (!stack.empty() && k > 0 && stack.back() > c) {
15             stack.pop_back();
16             k--;
17         }
18
19         stack.push_back(c);
20     }
21
22     while (stack.size() > remain) {
23         stack.pop_back();
24     }
25
26     while (stack.front() == '0' && stack.size() > 1) {
27         stack.pop_front();
28     }
29
30     string res;
31     for (char c: stack) {
32         res += c;
33     }
34
35     return res;
```

```
36 | }  
    | 37 |  
38 | int main() {  
39 |     string num;  
40 |     int k;  
41 |  
42 |     cin >> num >> k;  
43 |  
44 |     cout << solution(num, k);  
45 |  
46 |     return 0;  
47 | }
```